

SITUAÇÃO 1 · CUPCAKE ÁGIL

Planejamento revisado

Revisão do PIT I: escopo, modelagem UML, interface e banco de dados.

Ícaro Rocha · Projeto Integrador Transdisciplinar em Engenharia de Software II
cupcake-store.imrocha.dev · github.com/imrochadev/cupcake-agil

No PIT I eu planejei um aplicativo de venda de cupcakes. Neste documento eu volto a esse plano e mostro o que mudou quando ele virou um sistema de verdade. São quatro partes: o escopo (o que entrou e o que ficou de fora), os diagramas UML, as telas e o banco de dados.

> **Projeto:** Cupcake Ágil, Projeto Integrador Transdisciplinar em Engenharia de Software II.
> **Site:** cupcake-store.imrocha.dev
> **Código:** github.com/imrochadev/cupcake-agil

Sumário

1. Escopo revisado
2. Modelagem UML
3. Interface (IHC)
4. Banco de dados

1. Escopo revisado

Ponto de partida (PIT I)

No PIT I eu levantei os requisitos de um **aplicativo de venda de cupcakes** usando Scrum. O backlog ficou organizado assim: 5 épicos (CUPC-1 a CUPC-5) e 10 histórias de usuário (CUPC-6 a CUPC-15). Cada história tem os critérios que dizem quando ela está pronta. O trabalho foi dividido em três sprints:

- **Sprint 1, o básico:** cadastro, login e cardápio.
- **Sprint 2, a compra:** carrinho, pagamento e acompanhamento do pedido.
- **Sprint 3, o painel da loja:** gerenciamento da loja, favoritos, avaliação de produtos e histórico de pedidos.

O que mudou na revisão

Algumas ideias do PIT I mudaram quando o projeto saiu do papel. A tabela mostra cada mudança e o motivo.

Planejado no PIT I	O que foi feito	Por quê
Aplicativo para celular (Android e iOS)	**Site que funciona bem no celular**	Um site abre em qualquer celular sem instalar nada. Assim, os colegas que testaram a loja só precisaram de um link.
Pagamento de verdade	**Pagamento de mentira**, que se comporta como um real: confere o número do cartão, a validade e pode recusar	É um projeto da faculdade, então nenhum valor pode ser cobrado. Existe um cartão de teste que sempre é recusado, para mostrar o que acontece quando o pagamento dá errado.
Login com uma técnica que guarda a sessão no navegador de um jeito mais exposto	**Login com sessão protegida**, que nenhum código da página consegue ler	É mais seguro para um site. A troca não muda nada para quem usa.
Notificação no celular	**Aviso na tela e notificação do navegador** quando o pedido muda de etapa	Funciona sem instalar aplicativo. A tela do pedido confere a situação sozinha a cada 10 segundos.
Entrega acompanhada por uma equipe de verdade	**Entrega de mentira**: o pedido passa para a próxima etapa sozinho a cada 2 minutos	Assim quem testa vê o pedido chegar até "entregue" sem ninguém precisar mexer no painel.
Banco de dados em tabelas	**Banco de dados em documentos**	O modelo foi pensado primeiro em tabelas, como manda o material, e depois adaptado. A seção Banco de dados mostra o caminho.
Login só com e-mail e senha	E-mail e senha, **ou conta do Google ou do GitHub**	Menos coisa para preencher na hora de testar.

Rastreabilidade do backlog

Esta tabela liga cada item do backlog do PIT I ao que foi entregue.

Item	Prioridade	Situação
CUPC-1 Autenticação (épico)	Alta	Feito
CUPC-6 Cadastro de usuário	Alta	Feito
CUPC-7 Login no sistema	Alta	Feito
CUPC-2 Catálogo (épico)	Alta	Feito
CUPC-8 Visualização do catálogo	Alta	Feito
CUPC-3 Compras (épico)	Alta	Feito
CUPC-9 Carrinho de compras	Alta	Feito
CUPC-10 Pagamento	Alta	Feito (de mentira)
CUPC-4 Entregas (épico)	Média	Feito
CUPC-11 Rastreamento do pedido	Média	Feito
CUPC-5 Administração (épico)	Alta	Feito
CUPC-12 Gerenciamento administrativo	Alta	Feito
CUPC-15 Histórico de pedidos	Média	Feito
CUPC-13 Favoritos	Baixa	Ficou para depois
CUPC-14 Avaliação de produtos	Média	Ficou para depois
Formulário de avaliação do site (pedido no PIT II)	Não se aplica	Feito

Fora do escopo

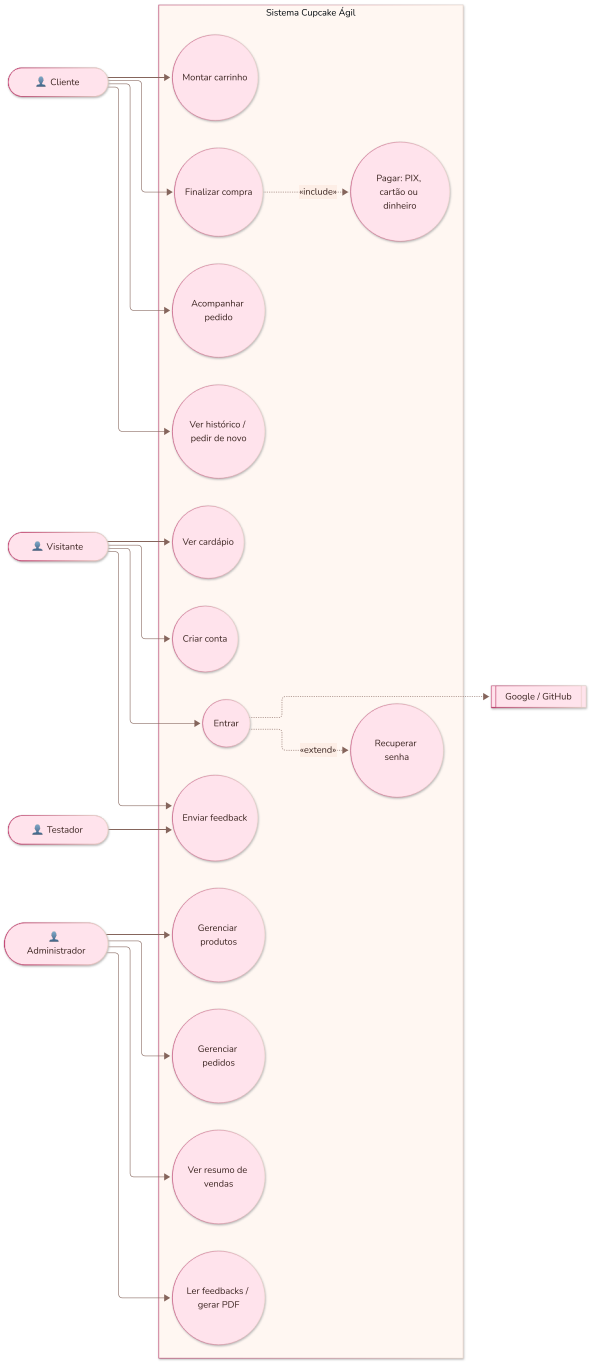
- Cobrança de verdade, nota fiscal e frete calculado pela distância. A entrega custa R$ 7,00 e é grátis a partir de R$ 80,00.
- Aplicativo publicado na loja do celular.
- Favoritos e avaliação de produtos, que são os próximos itens do backlog.

2. Modelagem UML

Os diagramas do PIT I foram revistos para mostrar o sistema como ele ficou.

Casos de uso

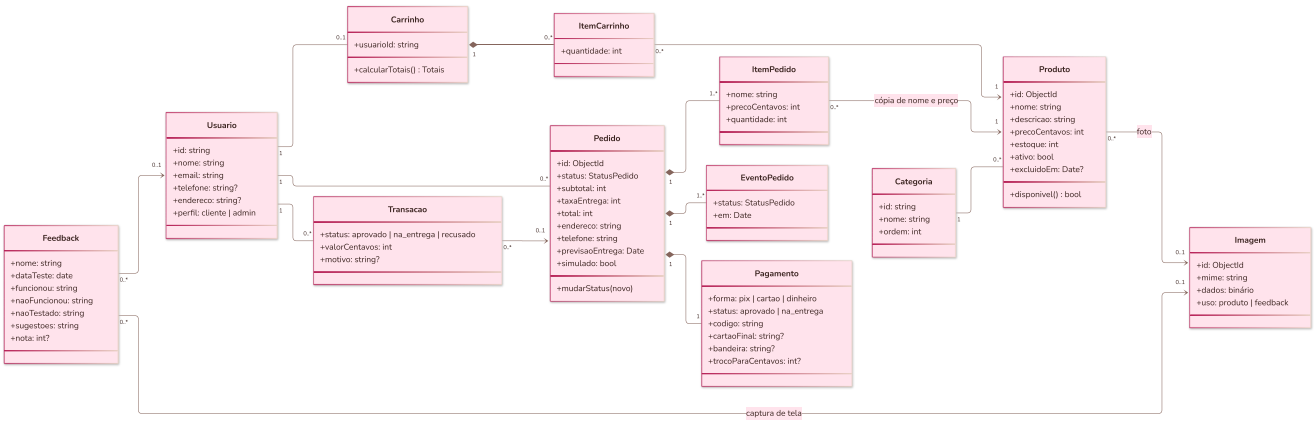
O diagrama mostra quem usa a loja e o que cada um consegue fazer.



O cliente é um visitante que já tem conta, então ele também faz tudo o que o visitante faz. O administrador também pode comprar como cliente.

Diagrama de classes

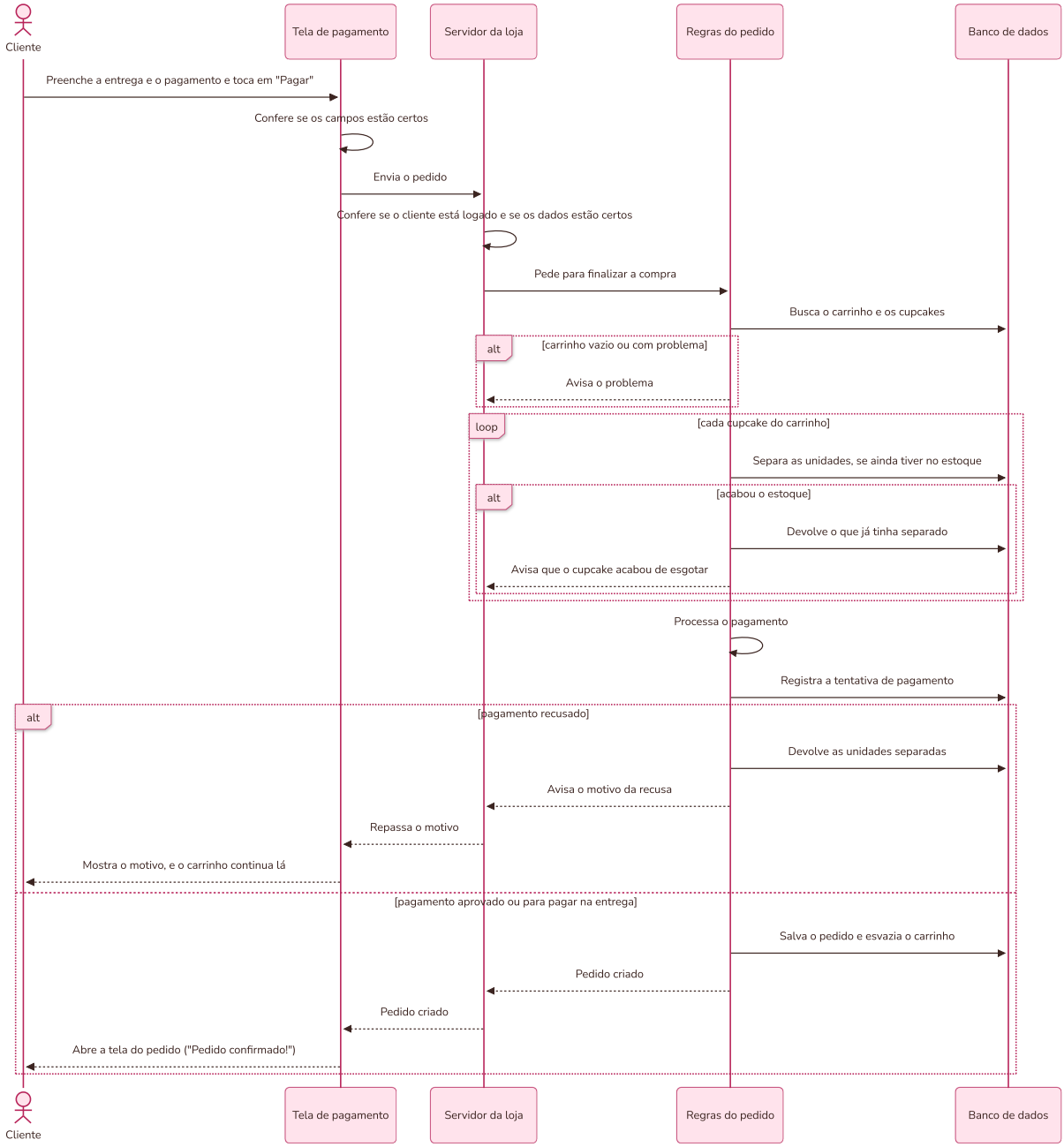
As principais partes do sistema, as informações de cada uma e como elas se ligam.



Os losangos (◆) marcam as partes que só existem dentro de outra. Os itens, as etapas e o pagamento fazem parte do pedido. O item do pedido guarda **uma cópia** do nome e do preço do cupcake, para o pedido não mudar se o cardápio mudar depois.

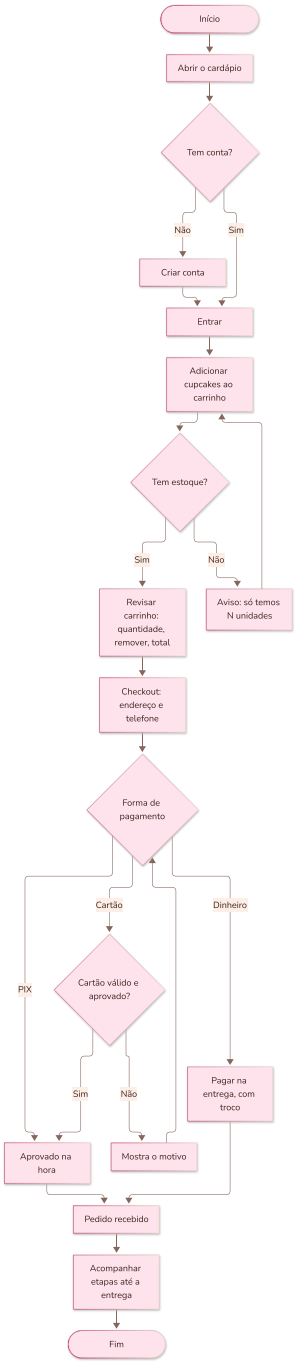
Sequência: finalizar a compra

Este diagrama mostra, em ordem, o que acontece quando o cliente aperta "Pagar".



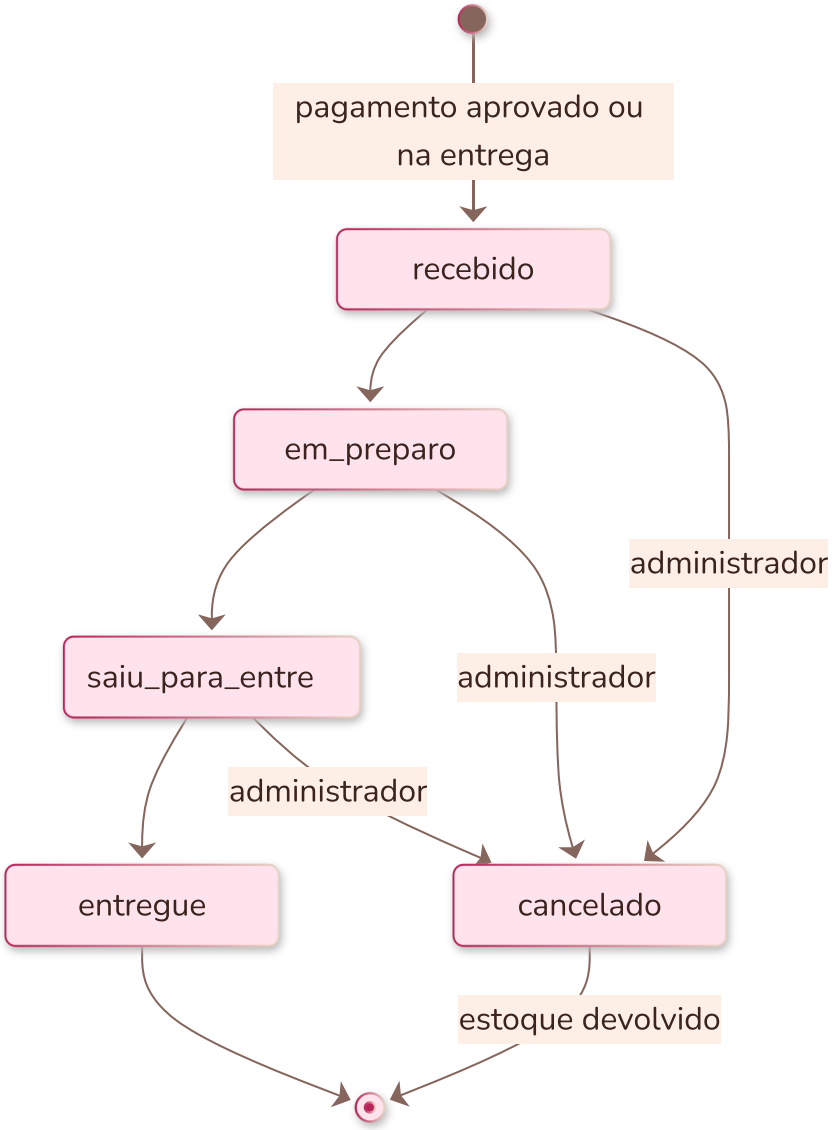
Atividades: jornada de compra

O caminho do cliente, do cardápio até a entrega.



Estados do pedido

Por quais etapas um pedido passa.



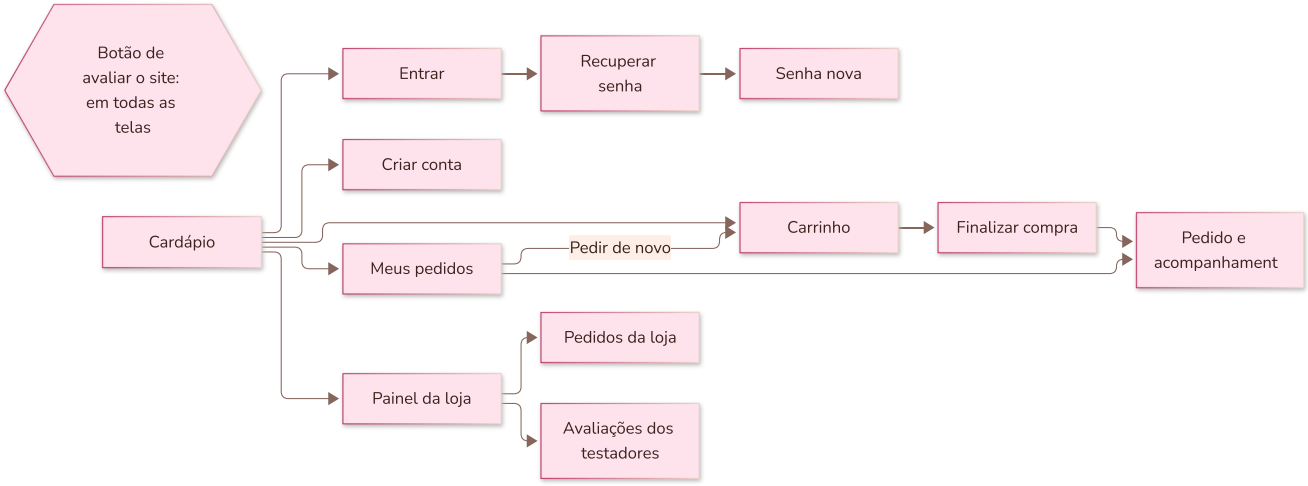
O administrador só avança uma etapa por vez e só cancela um pedido que ainda não foi entregue. Na entrega de mentira, o próprio sistema avança as etapas até o administrador mexer no pedido.

3. Interface (IHC)

O protótipo do PIT I virou as telas do site. As imagens do fim desta seção são da loja funcionando.

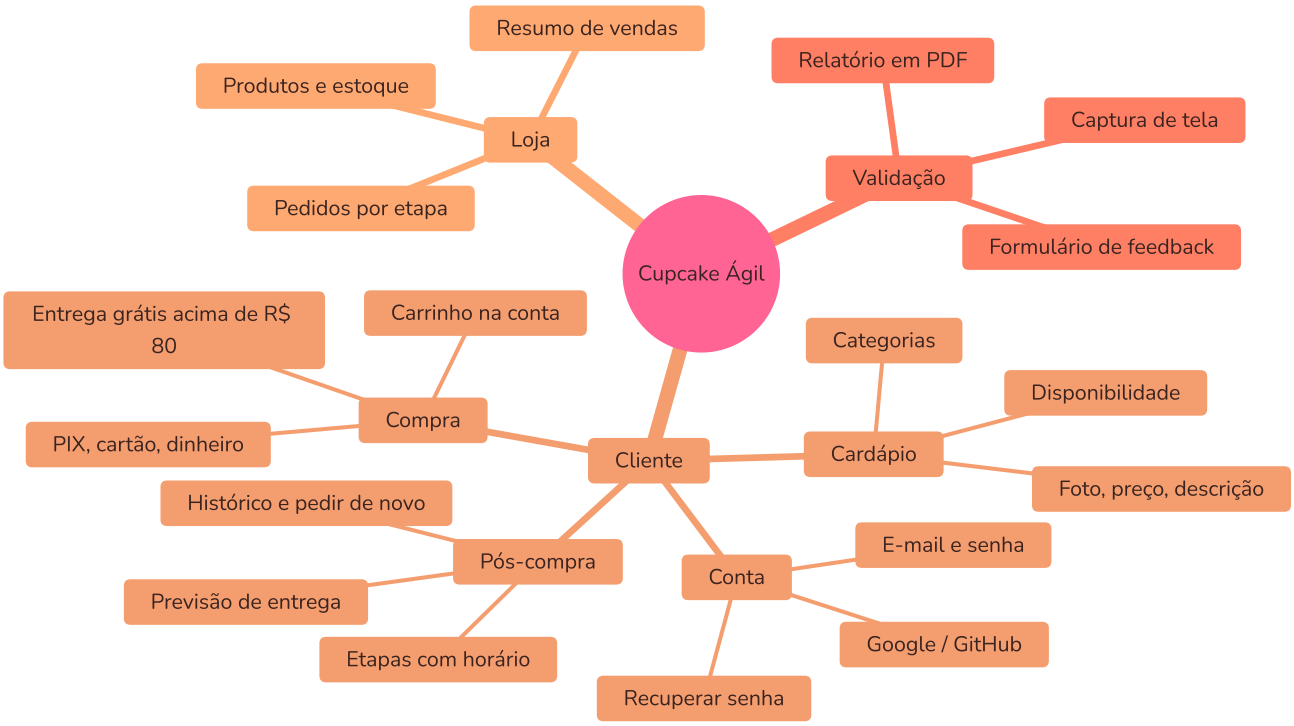
Mapa de telas

Como uma tela leva à outra.



Mapa conceitual

Tudo o que a loja tem, agrupado por assunto.



Princípios aplicados

O material da disciplina diz como uma boa interface deve ser. Cada princípio virou uma escolha no site:

Princípio	Como aparece no site
Clara	Cada tela tem um botão principal, na cor da loja, e o texto do botão diz o que ele faz: "Adicionar", "Pagar R$ 52,60", "Finalizar compra".
Consistente	Botões, campos e avisos têm o mesmo jeito em todas as telas.
Simples	Na hora de pagar, só aparecem os campos da forma de pagamento escolhida. O endereço e o telefone do cadastro já vêm preenchidos. Ao digitar o CEP, a rua, o bairro e a cidade aparecem sozinhos.
Controlada pelo usuário	Nada muda sem o cliente pedir. Excluir ou cancelar sempre pede confirmação.
Direta	Mudar a quantidade atualiza o total na hora, e o número do carrinho no topo sobe a cada cupcake adicionado.
Recuperável	Se o cartão for recusado, o carrinho e o que já foi digitado continuam lá. Um cupcake esgotado aparece com aviso em vez de sumir. "Pedir de novo" repete um pedido antigo.
Mostra onde está o erro	Se um campo estiver errado, a página rola até ele e deixa o cursor nele. No celular isso faz diferença, porque o erro costuma ficar fora da tela.

Princípio	Como aparece no site
Dá resposta	Todo botão mostra que está processando e depois mostra se deu certo ou errado. A tela do pedido avisa quando ele muda de etapa.
Bonita	Cores quentes (rosa e creme) e fotos dos cupcakes todas no mesmo estilo: fundo branco, mesmo ângulo e mesma luz.

Acessibilidade: todo campo tem um nome e o erro aparece junto dele. O leitor de tela (programa que lê a página em voz alta para quem não enxerga) entende o que cada botão faz. A janela de avaliação pode ser fechada com a tecla Esc. As cores do texto têm contraste suficiente com o fundo para quem enxerga pouco.

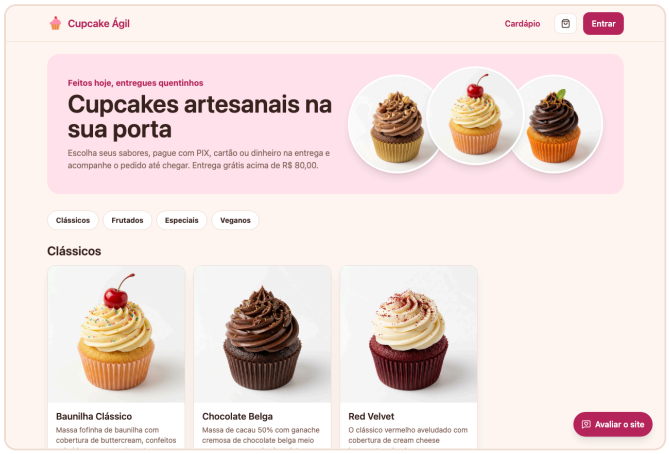
Mensagens de erro

Cada mensagem diz o que aconteceu e o que fazer. O campo com problema fica destacado.

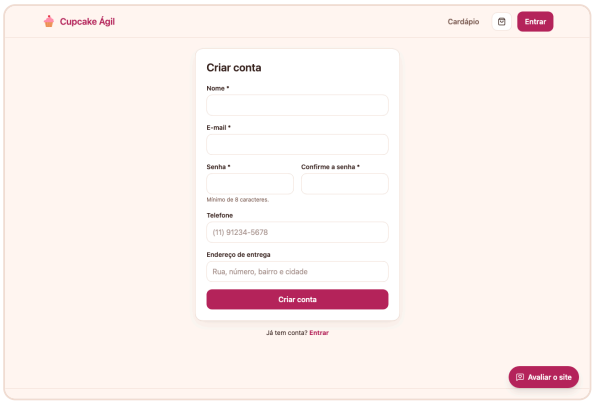
Situação	Mensagem
Login errado	E-mail ou senha incorretos.
Muitas tentativas	Muitas tentativas seguidas. Espere um minuto e tente de novo.
E-mail já cadastrado	Já existe uma conta com este e-mail. Entre ou recupere a senha.
Senha curta	A senha precisa de pelo menos 8 caracteres.
Telefone fora do formato	Telefone: use DDD + número, ex.: (11) 91234-5678.
Link de senha vencido	Este link expirou ou já foi usado.
Sem login numa página da conta	Entre na sua conta para continuar.
Mais do que tem no estoque	Só temos N unidade(s) de *Produto* agora.
Cupcake esgotado	*Produto* está esgotado no momento.
Estoque acabou durante a compra	*Produto* acabou de esgotar. Atualize o carrinho.
Número do cartão errado	Número do cartão inválido. Confira os dígitos.
Cartão vencido	Validade inválida ou cartão vencido (use MM/AA).
Cartão recusado	Pagamento recusado pela operadora (saldo insuficiente). Tente outro cartão ou outra forma de pagamento.
Troco menor que o total	O valor para troco precisa ser maior ou igual ao total do pedido.
Imagem inválida	Envie uma imagem PNG, JPEG ou WebP. / Imagem muito grande: o limite é 3 MB.
Painel sem permissão	Área restrita aos administradores da loja.
Loja fora do ar	A loja está fora do ar por um instante. Tente de novo em alguns segundos.

Situação	Mensagem
Sem internet	Sem conexão com a loja. Confira sua internet e tente de novo.

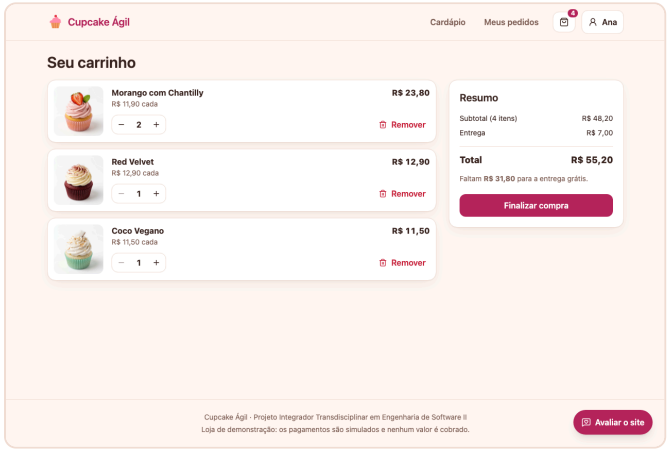
Telas



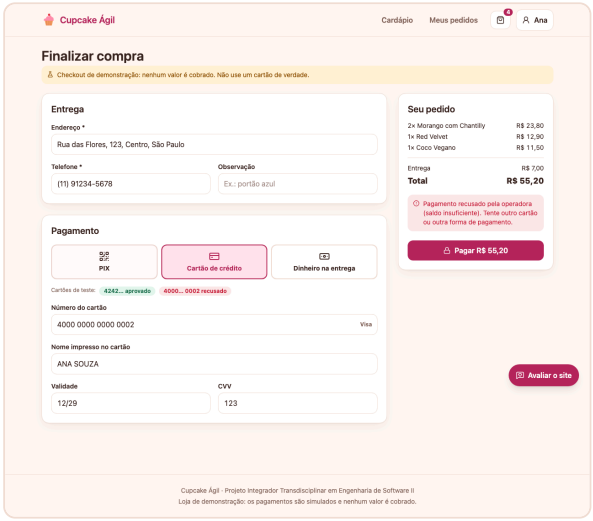
Cardápio



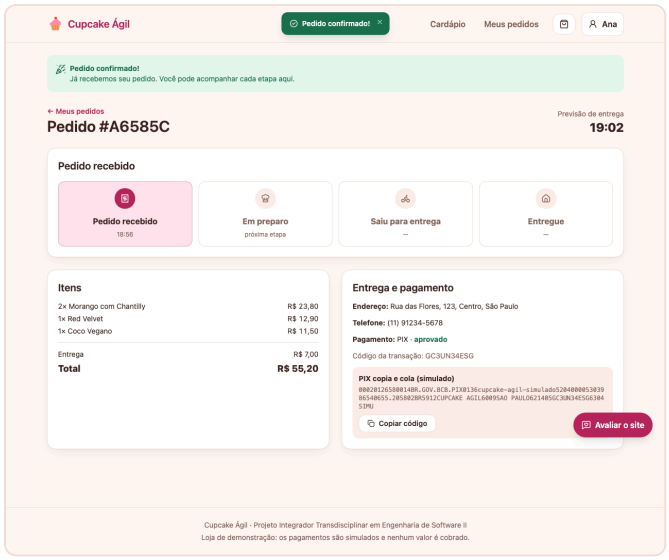
Cadastro



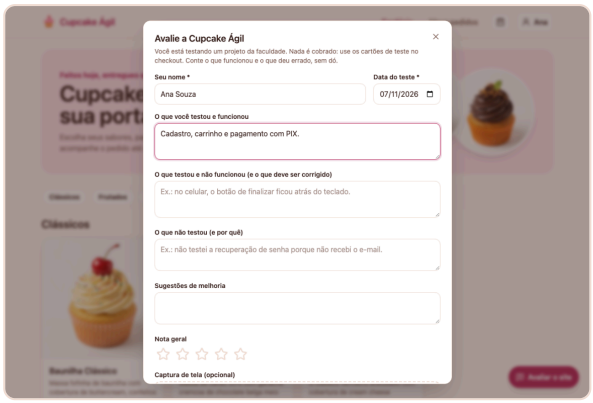
Carrinho



Pagamento: cartão recusado



Pedido confirmado e acompanhamento



Avaliação do site



Cardápio no celular

4. Banco de dados

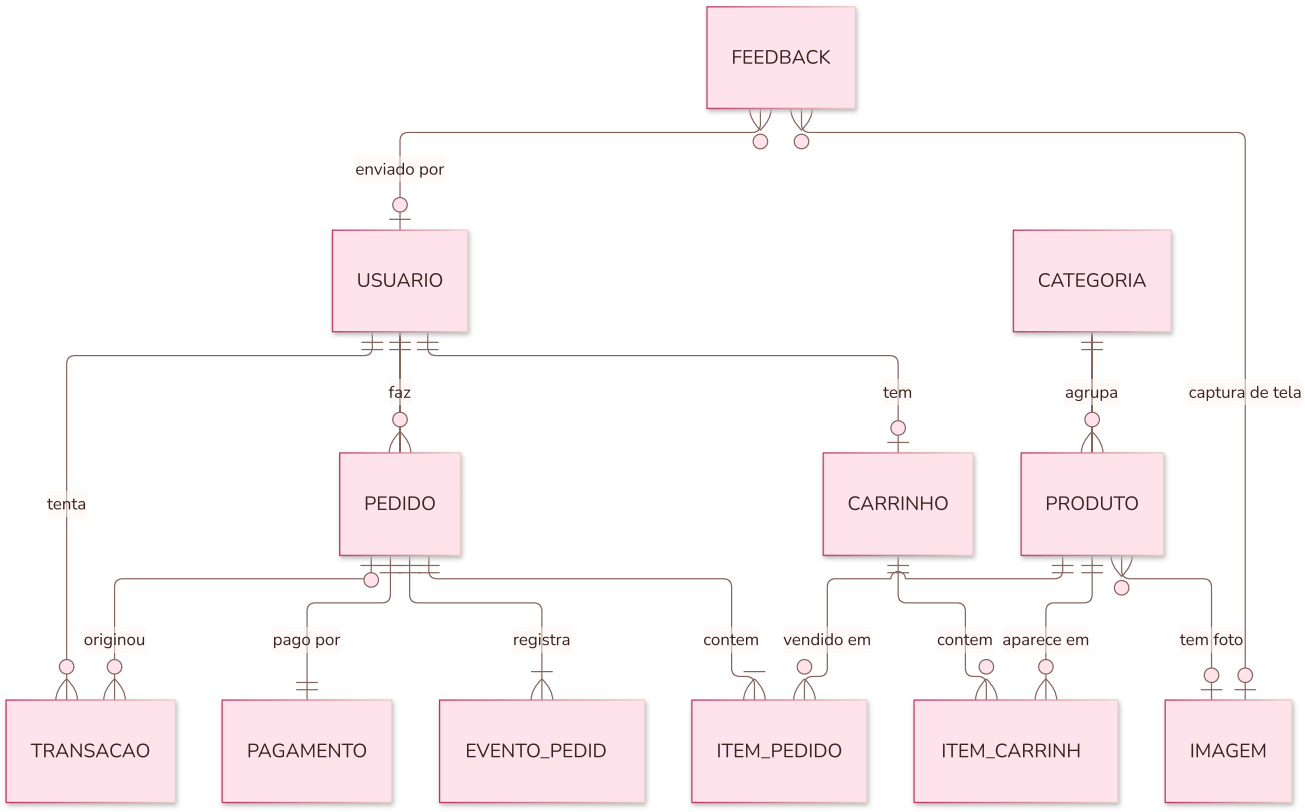
O banco de dados é onde a loja guarda tudo: contas, cupcakes, carrinhos, pedidos e avaliações. Ele foi desenhado em três passos, do mais geral para o mais detalhado:

1. **Conceitual:** quais "coisas" a loja guarda e como elas se ligam.
2. **Lógico:** cada coisa vira uma tabela, com as suas informações.
3. **Físico:** como essas tabelas ficam guardadas no banco escolhido.

O banco escolhido foi o **MongoDB**. Ele é muito usado, tem um plano gratuito na internet e é fácil de manter. Em vez de tabelas, ele guarda "documentos", que são parecidos com fichas.

1. Projeto conceitual

Cada caixa é uma coisa que a loja guarda. As linhas mostram como elas se ligam. Por exemplo: um usuário faz vários pedidos, e cada pedido tem vários itens.



2. Projeto lógico

Aqui cada coisa virou uma tabela. **PK** é a informação que identifica cada linha, como um RG. **FK** é uma informação que aponta para outra tabela. Por exemplo, o pedido guarda o identificador do usuário que comprou.

O modelo foi organizado para que nenhuma informação fique repetida em dois lugares. Assim, quando algo muda, só é preciso mudar num lugar.

```
USUARIO       (id PK, nome, email único, senha protegida, telefone, endereco, perfil,
criado_em)
CATEGORIA     (id PK, nome único, ordem)
IMAGEM        (id PK, tipo, dados, uso, criado_em)
PRODUTO       (id PK, categoria_id FK→CATEGORIA, imagem_id FK→IMAGEM, nome, descricao,
               preco_centavos, estoque, ativo, criado_em, atualizado_em, excluido_em)
ITEM_CARRINHO (usuario_id FK→USUARIO, produto_id FK→PRODUTO, quantidade)
PEDIDO        (id PK, usuario_id FK→USUARIO, status, subtotal, taxa_entrega, total, endereco,
telefone,
               observacao, previsao_entrega, simulado, criado_em)
ITEM_PEDIDO   (pedido_id FK→PEDIDO, produto_id FK→PRODUTO, nome, preco_centavos, quantidade)
EVENTO_PEDIDO (id PK, pedido_id FK→PEDIDO, status, em)
PAGAMENTO     (pedido_id FK→PEDIDO, forma, status, codigo, valor_centavos, cartao_final,
bandeira,
               troco_para_centavos)
TRANSACAO     (id PK, usuario_id FK→USUARIO, pedido_id FK→PEDIDO, forma, status,
valor_centavos,
               codigo, cartao_final, motivo, criado_em)
FEEDBACK      (id PK, usuario_id FK→USUARIO, imagem_id FK→IMAGEM, nome, data_teste, funcionou,
               nao_funcionou, nao_testado, sugestoes, nota, pagina, navegador, criado_em)
```

Uma repetição de propósito: o item do pedido guarda o nome e o preço do cupcake, mesmo eles já estando no cadastro do produto. É o valor **no dia da compra**. Se o preço subir amanhã, o pedido de hoje continua com o preço que o cliente pagou, igual a uma nota fiscal.

Dinheiro em centavos: todos os valores são guardados como número inteiro de centavos (R$ 12,90 vira 1290). Fazer conta com números quebrados no computador pode dar diferença de centavo, e com inteiros isso não acontece.

3. Projeto físico

Cada tabela virou uma **coleção** no MongoDB (uma pasta de fichas). A regra para decidir o que fica junto foi esta:

- **Fica dentro da ficha do dono** o que só existe junto com ele e sempre é lido junto. Os itens, as etapas e o pagamento ficam dentro do pedido. Os itens do carrinho ficam dentro do carrinho. Assim, uma única leitura traz o pedido inteiro, e ele é salvo de uma vez só.
- **Fica numa ficha separada, ligada por um identificador,** o que existe sozinho e é usado por várias outras fichas. Exemplos: a categoria e a foto de um cupcake, e o usuário de um pedido.
- **As contas de usuário** ficam em coleções próprias da ferramenta de login. A senha nunca é guardada como foi digitada: ela é embaralhada de um jeito que não dá para desfazer.

O banco recusa qualquer ficha que não siga as regras: campo obrigatório faltando, número negativo no estoque, etapa do pedido que não existe. Isso vale mesmo que alguém tente gravar direto no banco, sem passar pelo site.

Índices

Um índice funciona como o índice de um livro: ajuda o banco a achar as fichas rápido, sem olhar uma por uma.

Coleção	Organizado por	Para quê
Usuários	E-mail (sem repetir)	Login e impedir duas contas com o mesmo e-mail
Produtos	Categoria e nome	Montar o cardápio por categoria
Pedidos	Cliente e data	Histórico do cliente, do mais novo para o mais antigo
Pedidos	Etapa e data	Painel da loja filtrado por etapa
Transações	Cliente e data	Tentativas de pagamento do cliente
Avaliações	Data	Lista do painel

Duas pessoas comprando a última unidade

Se dois clientes tentarem comprar o último cupcake ao mesmo tempo, só um consegue. A loja só tira a unidade do estoque se ela ainda existir naquele exato momento, e o próprio banco garante isso. Se alguma coisa der errado depois (outro cupcake esgotou, o cartão foi recusado), as unidades separadas voltam para o estoque. Há um teste automático que confere esse caso.

4. Dicionário de dados

O dicionário descreve cada informação guardada. **Tipo** diz que tipo de informação é: texto, número inteiro, sim/não, data, identificador ou arquivo. **Obrig.** diz se a informação é obrigatória.

Categorias

Campo	Tipo	Obrig.	O que é
`_id`	texto (até 40)	sim	Identificador legível, ex.: `classicos`
`nome`	texto (até 40)	sim	Nome mostrado no cardápio
`ordem`	número inteiro	sim	Posição da categoria no cardápio

Produtos

Campo	Tipo	Obrig.	O que é
`_id`	identificador	sim	Identificador do cupcake
`nome`	texto (3 a 60)	sim	Nome do cupcake
`descricao`	texto (10 a 300)	sim	Descrição mostrada no cardápio
`categoriaId`	texto	sim	Categoria do cupcake
`precoCentavos`	número inteiro, no mínimo 1	sim	Preço em centavos (de R$ 1,00 a R$ 1.000,00)
`estoque`	número inteiro, no mínimo 0	sim	Unidades disponíveis. Zero quer dizer esgotado
`ativo`	sim/não	sim	Se aparece no cardápio
`imagemId`	identificador ou vazio	não	Foto do cupcake
`criadoEm`, `atualizadoEm`	data	sim	Quando foi criado e alterado pela última vez
`excluidoEm`	data ou vazio	não	Quando foi excluído. Some do cardápio e dos carrinhos, mas continua nos pedidos antigos

Imagens

Campo	Tipo	Obrig.	O que é
`_id`	identificador	sim	Identificador da imagem
`mime`	texto	sim	Formato da imagem: PNG, JPEG ou WebP
`dados`	arquivo (até 3 MB)	sim	A imagem em si. O formato é conferido pelo conteúdo, não pelo nome do arquivo
`uso`	`produto` ou `feedback`	sim	Foto de cupcake (todo mundo vê) ou print de uma avaliação (só o painel da loja vê)
`criadoEm`	data	sim	Quando foi enviada

Carrinhos

Campo	Tipo	Obrig.	O que é
`_id`	texto	sim	O usuário dono do carrinho (cada conta tem um)
`itens[].produtoId`	identificador	sim	O cupcake

Campo	Tipo	Obrig.	O que é
`itens[].quantidade`	número inteiro, no mínimo 1	sim	Quantas unidades. No máximo 20 por cupcake e nunca mais do que tem no estoque
`atualizadoEm`	data	sim	Última mudança no carrinho

Pedidos

Campo	Tipo	Obrig.	O que é
`_id`	identificador	sim	Os 6 últimos caracteres viram o número do pedido (#4ED212)
`userId`	texto	sim	Quem comprou
`status`	uma das etapas	sim	Recebido, em preparo, saiu para entrega, entregue ou cancelado
`itens[]`	lista (pelo menos 1)	sim	Cupcake, nome, preço e quantidade, copiados no momento da compra
`subtotal`, `taxaEntrega`, `total`	número inteiro	sim	Valores em centavos. A entrega custa R$ 7,00 e é grátis a partir de R$ 80,00
`endereco`	texto (até 300)	sim	Endereço de entrega numa linha: rua, número, complemento, bairro, cidade, estado e CEP
`telefone`	texto	sim	Telefone no formato (DD) 9XXXX-XXXX
`observacao`	texto (até 300)	sim, pode ser vazio	Observação do cliente, ex.: "portão azul"
`pagamento`	ficha	sim	Forma, situação, código, valor, final do cartão, bandeira e troco. Nunca guarda o número inteiro do cartão nem o código de segurança
`eventos[]`	lista	sim	Cada etapa do pedido com o horário
`previsaoEntrega`	data	sim	Horário previsto para a entrega
`simulado`	sim/não	sim	Se o pedido está andando sozinho (entrega de mentira) ou se a loja já assumiu
`criadoEm`	data	sim	Quando o pedido foi feito

Transações

Cada tentativa de pagamento, aprovada ou não.

Campo	Tipo	Obrig.	O que é
`_id`	identificador	sim	Identificador da tentativa

Campo	Tipo	Obrig.	O que é
`userId`	texto	sim	Quem tentou pagar
`pedidoId`	identificador ou vazio	não	O pedido criado. Fica vazio quando o pagamento foi recusado
`forma`	`pix`, `cartao` ou `dinheiro`	sim	Forma de pagamento
`status`	`aprovado`, `na_entrega` ou `recusado`	sim	Resultado
`valorCentavos`	número inteiro	sim	Valor em centavos
`codigo`	texto (10)	sim	Código da transação
`cartaoFinal`	texto (4) ou vazio	não	Só os 4 últimos números do cartão
`motivo`	texto ou vazio	não	Motivo da recusa
`criadoEm`	data	sim	Quando aconteceu

Avaliações (feedbacks)

Campo	Tipo	Obrig.	O que é
`_id`	identificador	sim	Identificador da avaliação
`nome`	texto (2 a 80)	sim	Nome de quem testou
`dataTeste`	data (AAAA-MM-DD)	sim	Dia do teste
`funcionou`	texto (até 2000)	*	O que testou e funcionou
`naoFuncionou`	texto (até 2000)	*	O que testou e não funcionou, e o que corrigir
`naoTestado`	texto (até 2000)	*	O que não testou, e por quê
`sugestoes`	texto (até 2000)	*	Sugestões de melhoria
`nota`	número de 1 a 5 ou vazio	não	Nota geral
`pagina`	texto	não	Tela em que a avaliação foi aberta
`navegador`	texto	não	Navegador e aparelho usados, para ajudar a repetir um problema
`imagemId`	identificador ou vazio	não	Print da tela
`userId`	texto ou vazio	não	Conta de quem avaliou, se estava logado
`criadoEm`	data	sim	Quando foi enviada

* Pelo menos um desses quatro campos precisa estar preenchido.

Contas de usuário

Estas coleções são criadas pela ferramenta de login.

Coleção	O que guarda
`user`	Nome, e-mail (sem repetir), perfil (cliente ou administrador), telefone, endereço e datas
`account`	Como cada pessoa entra: e-mail e senha (com a senha embaralhada), Google ou GitHub
`session`	Quem está logado agora e até quando
`verification`	Links de uso único, como o de criar uma senha nova (vale por 1 hora)